Università degli Studi di Trieste

Dipartimento di Ingegneria e Architettura

Corso di Laurea Magistrale
in Computer Engineering

CNN classifier

Computer Vision — Project Report

Daniyil Logvynenko

Anno Accademico 2025 – 2026

Contents

1 Motivation

I chose this project due to personal interest and especially due to the large diffusion of neural networks in the modern world. By implementing this project my goal was also to understand how neural networks are built and which algorithms they follow.

2 Introduction

In this project I was required to implement an image classifier based on convolutional neural networks with a provided dataset that contains 15 classes. My task was divided into three parts. First I had to build a shallow network from scratch according to a specific layout and some characteristics. In the second part I had to improve my network's performance by using some techniques suggested in the paper. In the last part I had to use a pre-trained network, like AlexNet, to improve the accuracy of my image classifier even more.

The performance of my neural network in each part was assessed by measuring loss and accuracy during training, for both the training set and the validation set. Afterwards, confusion matrix and the overall accuracy were computed on the test set.

3 Tools

As development environment for this project, I used my computer with Linux, VSCode and PyTorch (since it is a leading tool in the development of neural networks), with the dataset provided from Moodle, already divided in train and test sets.

4 Part 1: Building a Shallow Network from Scratch

At the beginning I was required to build a CNN from scratch following the layout shown in Table 1.

Table 1: Layout of the CNN used in Part 1

#	Type	Size
1	Image Input	$64 \times 64 \times 1$ images
2	Convolution	8 3×3 convolutions with stride 1
3	ReLU	
4	Max Pooling	2×2 max pooling with stride 2
5	Convolution	16 3×3 convolutions with stride 1
6	ReLU	
7	Max Pooling	2×2 max pooling with stride 2
8	Convolution	32 3×3 convolutions with stride 1
9	ReLU	
10	Fully Connected	15
11	Softmax	softmax
12	Classification Output	crossentropyex

Implementing this kind of CNN in PyTorch was straightforward, thanks to easy to understand syntax. The next step was to prepare and organize the data (images), so they could be processed by the CNN in the most efficient way. First of all I had to resize every image from the original size to 64×64 pixels and divide the test set into test and validation sets.

In order to make better use of computational power, architecture and memory of the GPU, I was required to feed images to the CNN in batches of 32 images. I also employed the Stochastic Gradient Descent optimizer after defining the CNN. In order to finish setting up the CNN, I had to set the initial bias values to 0 and use initial weights drawn from a Gaussian distribution with mean 0 and standard deviation 0.01.

To conclude, I chose appropriate values for the hyperparameters: a learning rate of `0.001` and number of epochs equal to 30, as it seemed a good compromise to observe how the CNN evolves.

4.1 Training Results

Figure 1 shows the loss and accuracy curves during training for both the training set and the validation set.

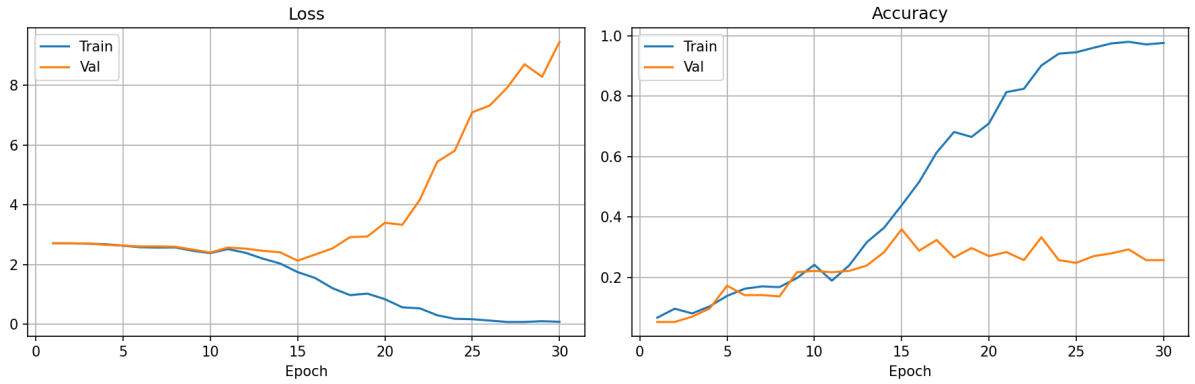


Figure 1: Loss and accuracy curves — shallow CNN (Part 1).

Looking at this graph we can easily observe that training accuracy reaches almost 100% and its loss is approaching zero. Meanwhile we cannot say the same about the validation set: accuracy does not reach even 40% and its loss is near 10. The answer is simple: our model is **overfitting**. The shallow CNN has too much capacity relative to the small size of the dataset. After roughly Epoch 15, instead of learning, the network starts simply memorizing training images, which explains the high validation loss.

I did not implement any explicit early stopping criterion. The model always trains for a fixed 30 epochs, but the system continuously monitors the validation accuracy. When evaluating on the final test set, we do not use the weights from Epoch 30 (validation loss: 9.46); instead, we load the saved weights from the epoch that achieved the highest validation accuracy (Epoch 15, peaking at 36.0%). This acts as an implicit form of **Early Stopping**, ensuring we evaluate the model at its peak generalization capability before memorization degrades performance.

4.2 Test Set Evaluation

Figure 2 shows the confusion matrix on the test set.

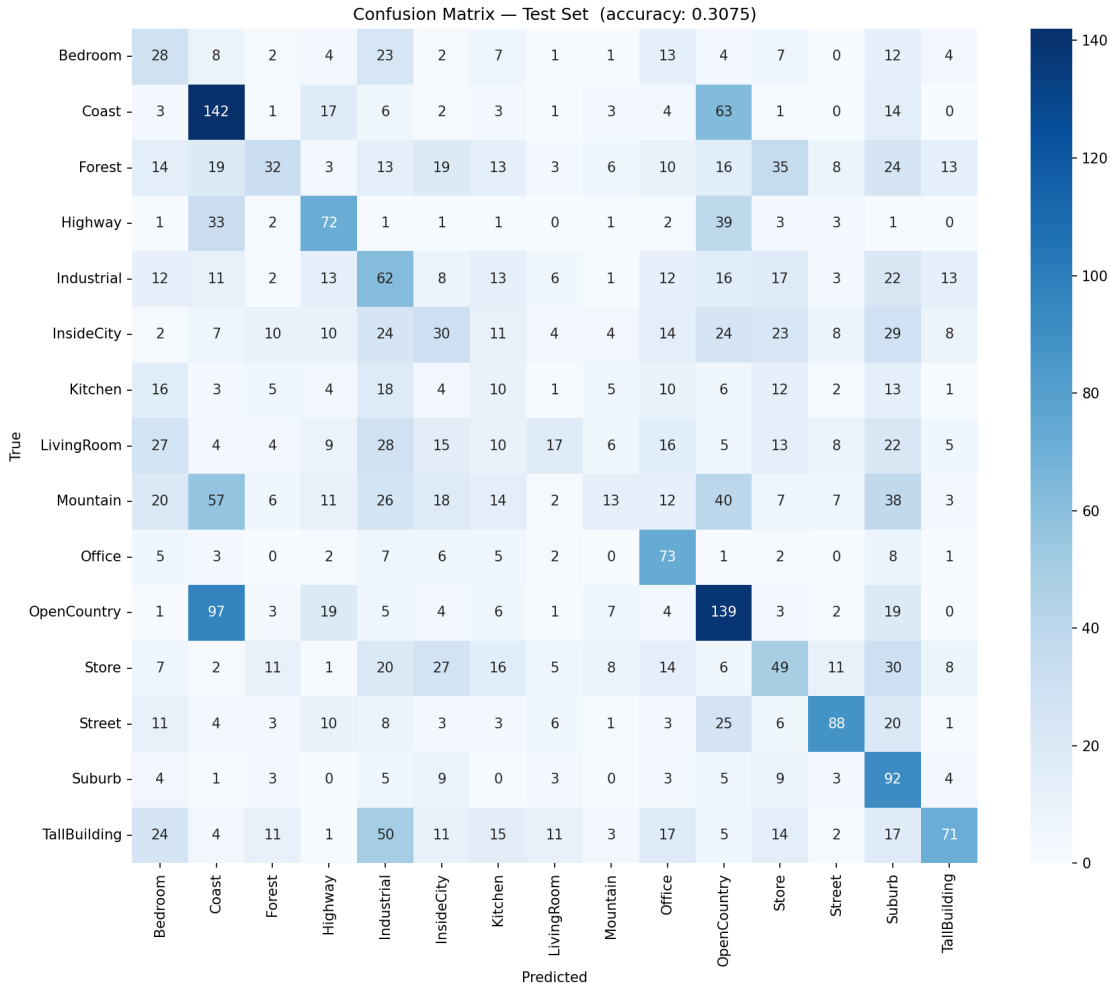


Figure 2: Confusion matrix on the test set — shallow CNN. Overall accuracy: 30.75%.

Overall Accuracy: The model achieved **30.8%** on the test set, which is definitely better than random guessing (6.6% for 15 classes). The confusion matrix reveals that the model struggles significantly with classes that are visually similar. For instance, the model frequently confuses *Coast* (Class 1) with *OpenCountry* (Class 10). This indicates that the shallow network lacks the representational capacity required to distinguish between visually similar landscape categories.

5 Part 2: Improving Test Accuracy

To improve the performance of my CNN, I decided to implement all the suggestions available in the reference paper.

- **Data Augmentation**: added `transforms.RandomHorizontalFlip(p=0.5)` to the training set transformation pipeline.

- **Batch Normalization**: added `nn.BatchNorm2d()` layers before each ReLU activation.
- **Dropout**: added a dropout layer at the end of the network with `self.dropout = nn.Dropout(p=0.58)` (the value 0.58 was chosen because it worked the best for my CNN).

5.1 Training Results

Figure 3 shows the updated training curves.

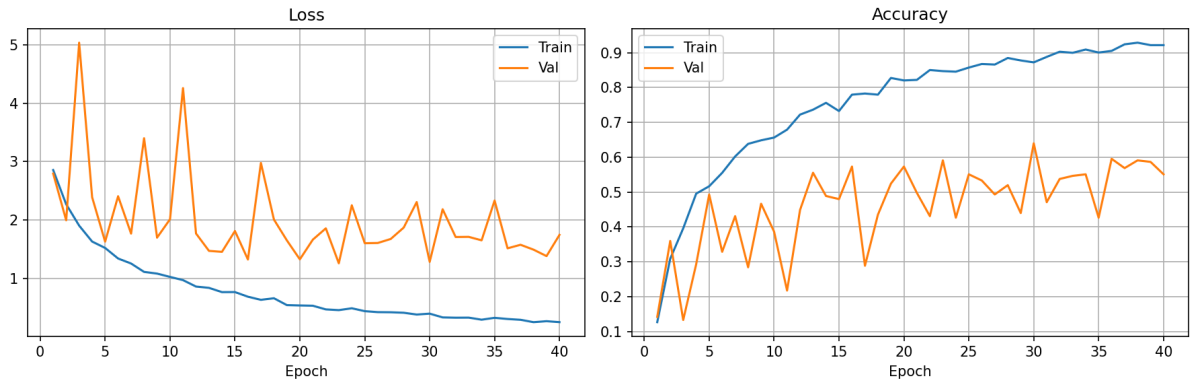


Figure 3: Loss and accuracy curves — improved CNN with Dropout, Batch Normalization and Data Augmentation (Part 2).

Training Set: Unlike the previous model that reached 97% of accuracy, the training accuracy here climbs much more gradually, finishing at 87.2%. This is a highly positive sign.

Validation Set: This is the real triumph. In the previous model, the validation accuracy collapsed after Epoch 15 and the validation loss exploded to 9.46. Here, the validation holds strong: the validation accuracy reaches its peak of **64.0%** at the very last epoch (Epoch 30) and the loss stabilizes around 1.28.

The introduction of Dropout and Batch Normalization alongside Data Augmentation drastically improved the severe overfitting observed in the baseline model. The model is successfully learning visual features rather than just memorizing.

5.2 Test Set Evaluation

Figure 4 shows the confusion matrix on the test set.

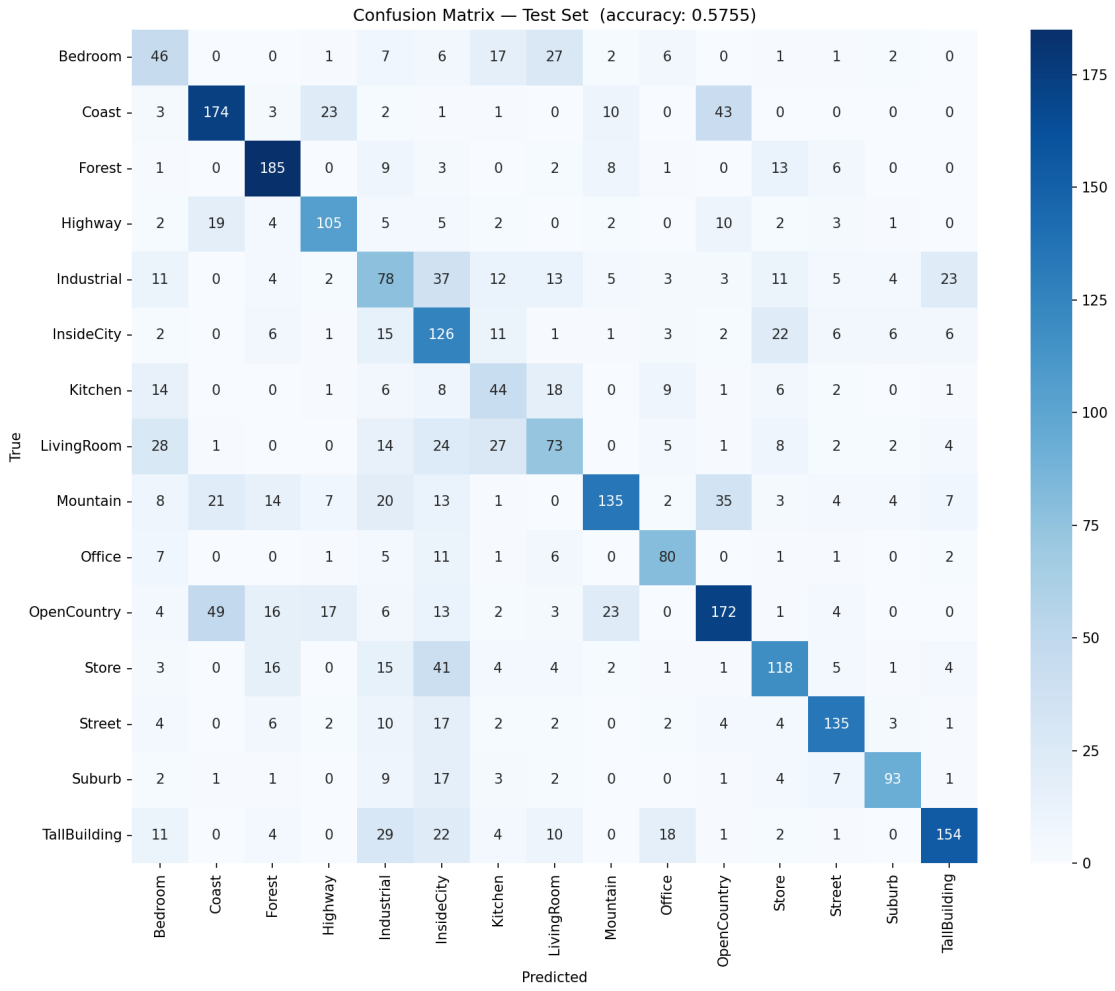


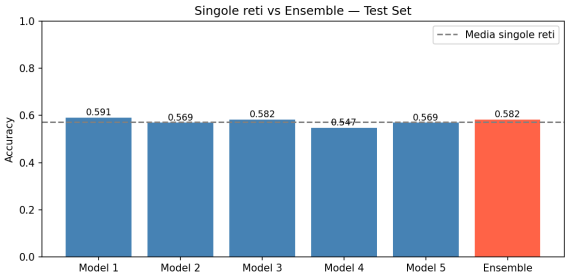
Figure 4: Confusion matrix on the test set — improved CNN. Overall accuracy: 57.55%.

The overall accuracy on the test set reached **57.6%**. The network now excels at distinguishing categories such as Forests, Coasts, and Open Country. The remaining errors are diffused among classes that are visually very similar (e.g., *Bedroom* vs. *Living Room*).

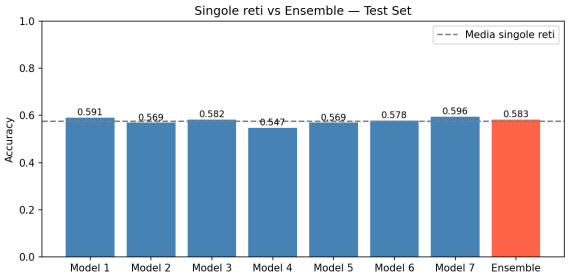
After some additional hyperparameter tuning (learning rate, kernel size, minibatch size, weight regularization), it was found that most changes either worsened performance or improved it by very little; the Adam optimizer did not bring any improvements. Therefore, the same hyperparameters from Part 1 were kept.

5.3 Ensemble Learning

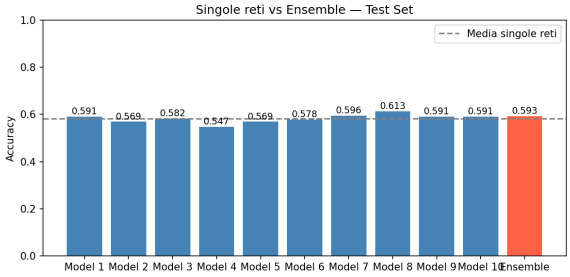
As a last step, an ensemble of independently trained networks was created. The arithmetic average of the output probability vectors was used to assign the final class. Ensembles of 5, 7, and 10 networks were evaluated, each trained with a different random seed to ensure diversity.



(a) 5-model ensemble



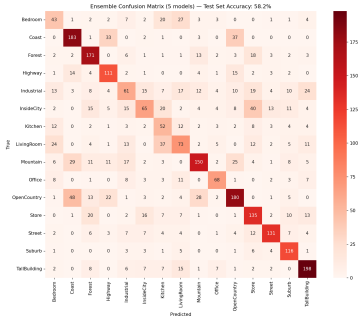
(b) 7-model ensemble



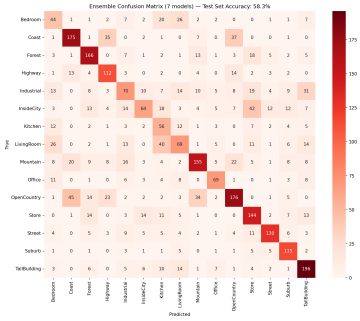
(c) 10-model ensemble

Figure 5: Accuracy comparison: individual models vs. ensemble on the test set.

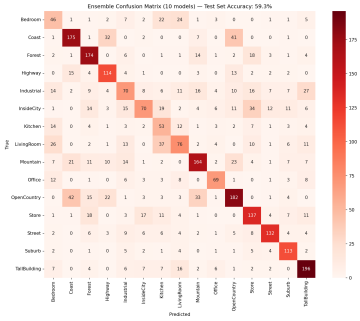
The ensemble confusion matrices are shown in Figure 6.



(a) 5 models — 58.2%



(b) 7 models — 58.3%



(c) 10 models — 59.3%

Figure 6: Ensemble confusion matrices on the test set.

The results are summarized below:

- **5-model ensemble**: Single models avg (val) 57.1% → Ensemble (test) 58.2% | Improvement: +1.0%
- **7-model ensemble**: Single models avg (val) 57.5% → Ensemble (test) 58.3% | Improvement: +0.7%
- **10-model ensemble**: Single models avg (val) 58.2% → Ensemble (test) 59.3% | Improvement: +1.1%

The ensemble guarantees a stable result that is consistently higher than the arithmetic average of the individual networks. Moving from 1 to 5 models provided a decent boost;

moving from 5 to 10 models doubled the training time, but yielded only an additional 1.1% in accuracy. Ensembles are powerful, but expensive in terms of memory, time and speed.

6 Part 3: Using a Pre-trained Network (Transfer Learning)

In the last part, transfer learning based on AlexNet was explored in two ways.

6.1 AlexNet with Frozen Weights (Fine-tuning Last Layer)

In the first case it was required to freeze the weights of all the layers but the last fully connected layer and fine-tune the weights with the same train and validation sets employed before. To implement something like that, it was needed to replace CNN from Part 2 with AlexNet, in the following manner:

Listing 1: AlexNet freeze setup in PyTorch

```
model = models.alexnet(weights=models.AlexNet_Weights.IMAGENET1K_V1
    )

# Freeze ALL parameters
for param in model.parameters():
    param.requires_grad = False

# Replace ONLY the last FC layer
model.classifier[6] = nn.Linear(4096, NUM_CLASSES)
model = model.to(device)
```

6.1.1 Training Results

Figure 7 shows the learning curves.

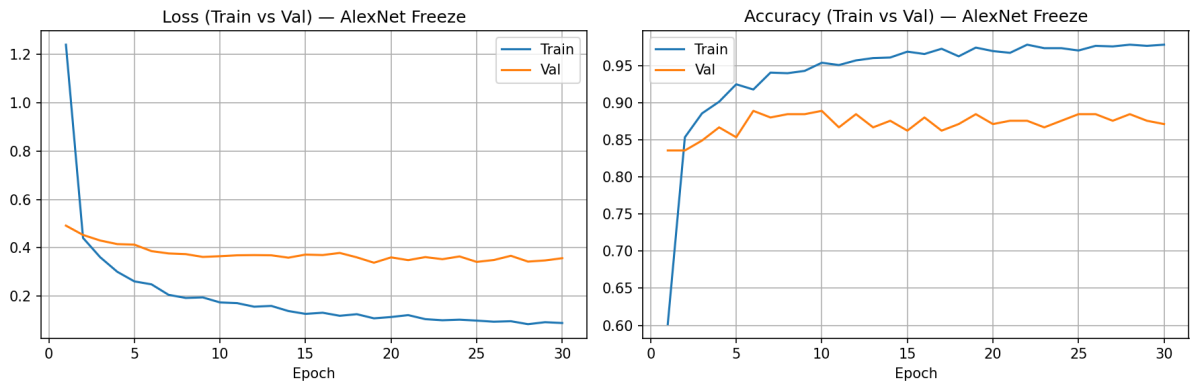


Figure 7: Loss and accuracy curves — AlexNet (frozen weights, fine-tuned last layer).

The learning curves demonstrate the immense advantage of Transfer Learning over training from scratch. At Epoch 1, unlike the previous shallow CNNs that began with near-random guessing, the AlexNet model immediately achieves a validation accuracy of over

83%. The validation curves remain highly stable: the validation loss plateaus around 0.35 and avoids the explosive up and down fluctuations that characterized the severe overfitting in earlier experiments.

6.1.2 Test Set Evaluation

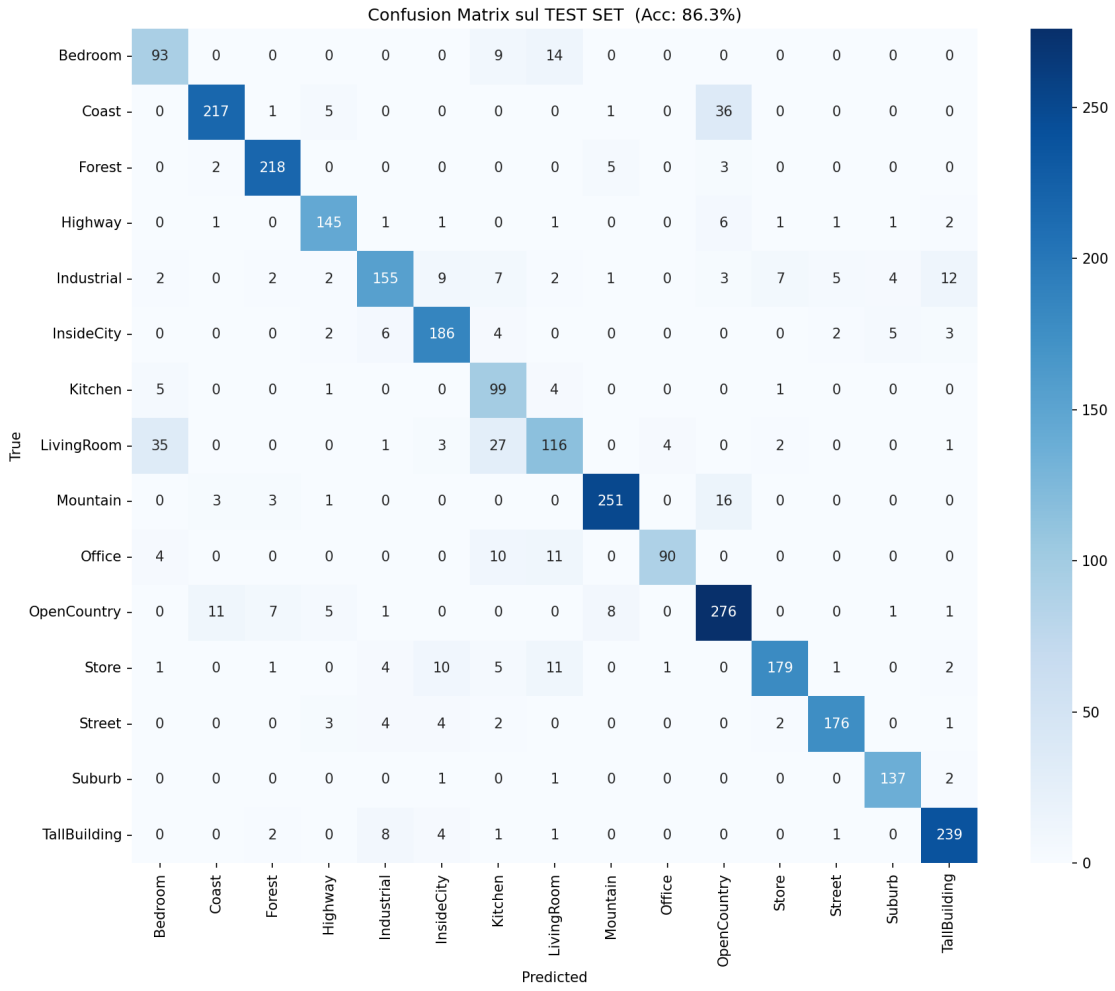


Figure 8: Confusion matrix on the test set — AlexNet frozen weights. Accuracy: 86.3%.

The overall accuracy on the test set reached outstanding **86.3%** , as we see AlexNet is very effective at recognising different classes, with only minor mistakes caused by visually very similar images.

6.2 AlexNet as Feature Extractor + Linear SVM

In the second approach, AlexNet was used as a feature extractor combined with Linear SVM predictor. To implement that, the last classification layer was replaced with an identity mapping:

Listing 2: Replacing the classifier head with Identity

```
model.classifier[6] = nn.Identity()
```

Afterwards each image is passed through AlexNet and, instead of a 15-class prediction, a **4096-dimensional feature vector** is returned. This vector is then fed to a multiclass linear SVM, defined like that:

Listing 3: Multiclass linear SVM (OVO)

```
svm = SVC(kernel='linear', decision_function_shape='ovo',
    random_state=SEED)
svm.fit(X_train, y_train)
predictions = svm.predict(X_test)
```

6.2.1 Results

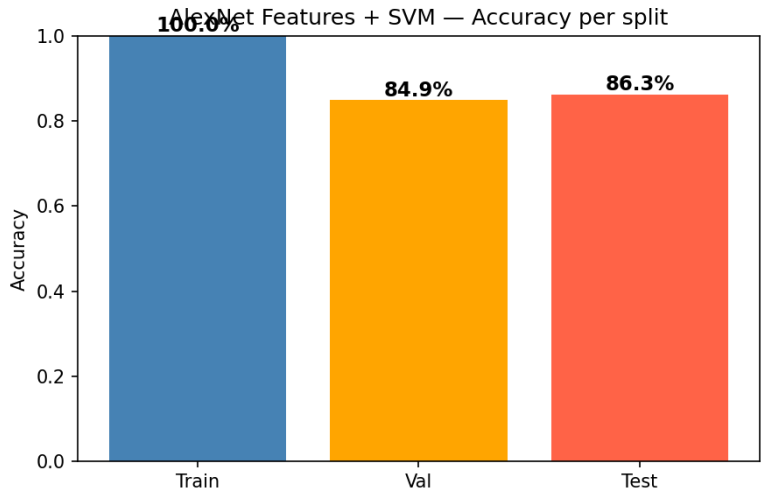


Figure 9: AlexNet Features + Linear SVM — accuracy per split.

Seeing a 100% score on the training set might raise alarm about overfitting, but for SVMs operating on deep feature vectors this is mathematically normal behaviour. In AlexNet's 4096-dimensional space, the training points are so well separated that the SVM finds hyperplanes that divide them without error. This is not a problem because validation (84.9%) and test (86.3%) scores are both high.

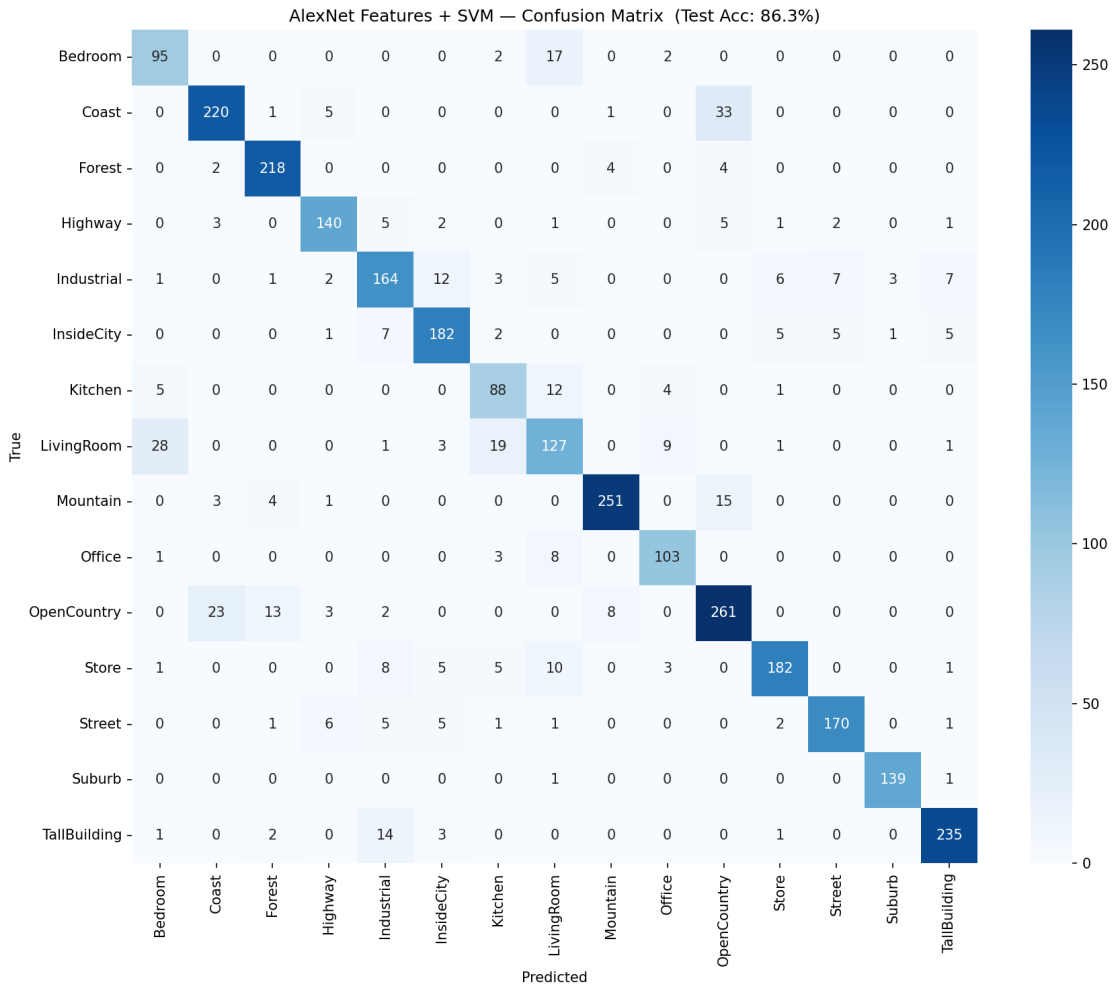


Figure 10: AlexNet Features + SVM — confusion matrix on the test set. Accuracy: 86.3%.

A frozen network coupled with a multiclass linear SVM (OVO) is as powerful as fine-tuning the final layer with SGD. The SVM's dividing hyperplanes do not merely memorise the images, but capture the logical boundaries between rooms and landscapes.

7 Final Remarks

This project highlighted the challenges of image classification on small datasets and the techniques used to overcome them. Initially, training a shallow CNN from scratch led to severe overfitting. Applying regularisation techniques as Dropout, Batch Normalisation and Data Augmentation successfully pushed the test accuracy to ≈58%; Ensemble Learning provided further stabilisation (≈59%), but the sheer complexity of raw pixel classification remained a bottleneck.

The definitive breakthrough was achieved through **Transfer Learning**. By leveraging the pre-trained weights of AlexNet, the model did not need to learn basic visual features from scratch. This approach not only drastically reduced training instability but also pushed the test accuracy to **86.3%**. The remaining misclassifications are not random but arise from genuine visual similarity between classes (e.g., Bedroom vs. Living Room or Coast

vs. Open Country). This project demonstrated that for datasets with limited samples and low resolution, using a robust pre-trained CNN is vastly superior than training custom networks from the ground up.

8 Conclusion

This project proved to be very interesting, as it provided practical understanding of theoretical concepts that might otherwise remain unclear. Through building and using CNNs and SVMs, I gained hands-on experience that can be applied in future projects, very probably in other courses.

Disclaimer

The code used in this project was largely written with the help of Large Language Models (LLMs), due to my limited knowledge of the PyTorch framework. Despite this, I am able to understand the implemented code. For this reason, design choices, results and any potential errors or inaccuracies present in this project are my responsibility.

References

- Class Slides [available on Moodle]